

BR-Bench: A Benchmark Linking Continuous Integration Changes to Fault-Revealing Tests

Deepanshu*, Sanskriti Singh*, Prisha Vadhavkar*, Dr. Yoga Raja C A†

*School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore, Tamil Nadu, India

{deepanshu.2023, sanskriti.singh2023, prisha.vadhavkar2023}@vitstudent.ac.in

†Assistant Professor Sr. Grade 1, Department of Information Technology

School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore, Tamil Nadu, India

yogaraja.ca@vit.ac.in

Abstract—Continuous Integration (CI) services execute automated test suites to detect regressions introduced by pull requests. While industrial platforms at Meta and Google leverage predictive models and large-scale workload analyses, the research community lacks public, reproducible benchmarks linking pull-request modifications directly to individual failing tests. We present BR-BENCH, an open dataset linking code changes to fault-revealing tests mined from GitHub Actions logs across Java and Python open-source repositories. A test is defined as fault-revealing if and only if it fails at the head commit, passes or is absent in the resolved base run, and does not exhibit flakiness across runs on the same commit. The strict benchmark split contains 762 pull-request instances, 4,168 fault-revealing labels, and 2,466 distinct tests across 37 repositories. We evaluate test-to-file binding against pinned commit trees, achieving 94.29% (5,643/5,985) combined binding and 63.83% (3,820/5,985) full-confidence fully qualified name binding. In an illustrative application evaluating predictive test selection proxies at $k = 10$, historical failure frequency achieves a micro recall of 39.44% (519/1,316) whereas all-partner co-change file mining achieves only 4.64% (61/1,316). We provide complete extraction pipelines without language model dependencies, processed release tables, and offline reproduction via a single command.

Index Terms—continuous integration, test selection, regression testing, mining software repositories, fault revelation

I. INTRODUCTION

Modern software development relies on continuous integration (CI) workflows to catch defects before pull requests merge into shared branches. In CI pipelines, every code modification triggers test suites that consume substantial time and energy. Predicting which individual tests are likely to fail on a given pull-request diff would let teams run failure-prone tests first and conserve compute.

Prior research addresses neighboring questions rather than empirical test-failure prediction. Change impact analysis (CIA) predicts co-changed program entities using version-control history [1], [2], capturing editing coupling rather than defect propagation. Regression test selection (RTS) tools, such as Ekstazi [3] and STARTS [4], compute safe over-approximations via dynamic file or static class dependencies. In contrast, industrial deployments at Meta [5] and Google [6] run on proprietary monorepos, leaving underlying failure datasets inaccessible to researchers.

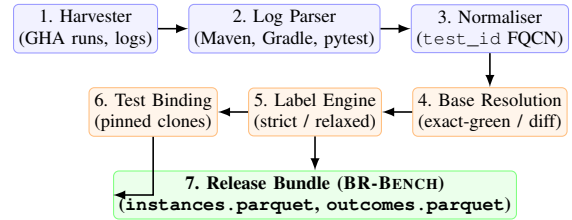


Fig. 1. The BR-BENCH construction pipeline. Raw CI logs and metadata are harvested, parsed into canonical test identifiers, matched against resolved base runs, filtered for flakiness, bound to source trees, and emitted as Parquet tables.

Existing open benchmarks do not fill this void: RTPTorrent [7] targets Travis CI job-level prioritization for Java, while GHALogs [8] archives raw GitHub Actions logs without test-level extraction, source binding, or flake filtering.

To bridge this divide, we present BR-BENCH, an open benchmark connecting pull-request file changes directly to the individual test cases that failed because of them. Mined from real GitHub Actions pipelines across 76 swept repositories, BR-BENCH extracts structured test outcomes from Gradle, Maven, and pytest execution logs. We resolve base runs on repository histories, filter same-commit test flakiness, and bind test identifiers back to repository source paths.

II. DATA SOURCE AND COLLECTION METHOD

The construction pipeline of BR-BENCH consists of seven reproducible stages (Fig. 1): repository selection, metadata harvesting, log parsing, identifier canonicalisation, base-run resolution, test-to-file binding, and release bundling.

A. Repository Sampling and Harvesting

We selected candidate repositories from the SEART GitHub platform [9]. Starting from an export of 3,671 open-source repositories, we retained 2,646 repositories active on GitHub Actions with at least 100 workflow runs in the preceding 90 days. We identified 2,333 repositories executing test-intent workflows (excluding linting or deployment tasks). We sampled 300 repositories into a balanced evaluation frame across Java and Python. The funnel counts 76 swept repositories and 165,349 runs (`instances_raw`); the harvest corpus has

77 repositories and 175,038 runs (`corpus_stats.md`). For each workflow run, the harvester downloaded run metadata, job step statuses, check-run annotations, and raw textual execution logs via the GitHub REST API.

B. Log Parsing and Test Identifier Normalisation

Because GitHub Actions treats test execution as unstructured stdout and stderr streams, test verdicts must be parsed from job logs. We built deterministic line parsers for three major test runners: Maven Surefire/Failsafe, Gradle Test Runner, and pytest. The parsers extract test outcomes (pass, fail, error, skip) and execution durations without relying on external large language models (LLMs). Each parsed test is mapped to a canonical identifier (`src/parse/test_ids.py`): for Java, the format is `fully.qualified.Class#method`; for Python, the format is `path::name`. Bare class identifiers are disambiguated using package declarations recovered from logs where available.

C. Base-Run Resolution and Fault Revelation

A test failure at a pull request’s head commit does not imply that the pull request caused the failure; the test may have been broken in the upstream base branch. To establish causality, every failed head run must be paired with an appropriate base run. For each pull-request run on head commit H targeting base branch commit B , we search repository history for a workflow run on B executing the identical workflow. If a run on B exists and completed with conclusion success, we record an `exact_green` resolution. If no run exists on B , we traverse ancestor commits along the default branch within a maximum distance of 10 commits. In total, base resolution records five mutually exclusive statuses: `exact`, `exact_green`, `ancestor` (max distance 10), `branch_prior`, and `no_base`. Under Invariant 6, runs resolving to `no_base` emit zero labels.

The benchmark defines three nested splits (`src/label/fault_revealing.py`): (1) `all` contains every (run, test) head failure in workflow runs with a usable base and parsed head test output; (2) `relaxed` removes tests that also fail in the resolved base run ($T_{\text{head_fail}} \setminus T_{\text{base_fail}}$); and (3) `strict` equals relaxed minus same-SHA flips (46/4,214 labels removed, 1.09%) to prune non-deterministic flakiness (T_{flaky}), yielding 4,168 strict positive labels:

$$T_{\text{reveal}} = (T_{\text{head_fail}} \setminus T_{\text{base_fail}}) \setminus T_{\text{flaky}}. \quad (1)$$

D. Flakiness Filtering and Test-to-File Binding

Flaky tests flip their status non-deterministically without code changes [10]. We detect same-SHA flips across repeated workflow executions on the exact same commit hash. Across 11,557 evaluated (head SHA, workflow, test) groups, 62 instances (0.54%) flipped, leading to the removal of 46 flaky labels from the relaxed set (1.09% of 4,214 labels) to form the strict set.

To enable change impact analysis, canonical test identifiers must bind to physical test source files. We bind identifiers against pinned repository clones recorded in

TABLE I
KEY SCHEMA FIELDS OF BR-BENCH RELEASE TABLES (V0.2)

Field	Type, Nullable, and Description
<i>instances.parquet</i> (165,349 rows, one per workflow run)	
<code>instance_id</code>	String, no; PK: sha256(repo head_sha run_id)
<code>repo, language</code>	String, no; repository (owner/name) and language (Java/Python)
<code>head_sha, base_sha</code>	String, no; commit SHA at head and target on base branch
<code>run_started_at</code>	Timestamp, no; UTC run start time (lossless cast from ISO-8601)
<code>base_run_id</code>	Int64, yes; resolved base run ID (7,061 filled, 4.27%)
<code>base_status</code>	Enum, no; exact_green, exact, ancestor, branch_prior, no_base, not_attempted
<code>base_run_distance</code>	Int32, yes; commit distance to base run (6,025 filled, max 10)
<code>Changeset diffs</code>	Int32 / Int64 / Bool / List, yes; <code>n_files_changed</code> , <code>n_lines_changed</code> , <code>touches_*</code> , <code>is_docs_only</code> , <code>changed_files</code> paths (null for 17,364 runs without diff)
<code>author_login</code>	String, no; 16-hex HMAC-SHA256 pseudonymised hash
<i>outcomes.parquet</i> (12,766 rows; grain: instance × test × split)	
<code>run_id, test_id</code>	Int64 / String, no; FK to instances and canonical test ID
<code>split</code>	Enum, no; benchmark split (all, relaxed, strict)
<code>test_file</code>	String, yes; bound test source path (4,229/4,384 pairs = 96.5%)
<code>label_source, status_head</code>	Enum, no; source (log) and head verdict (fail/error)
<code>parser_confidence</code>	Float, no; deterministic parser confidence metric
<code>n_job_rows</code>	Int32, no; matrix legs aggregated into pair (1 for 690 pairs)
<code>binding_status, confidence</code>	Enum / Float, yes; status (exact, etc.) and confidence (1.0 FQCN, 0.5 base)
<code>duration_s, hash</code>	Float / String, yes; test duration (57.1%) and SHA-256 error hash (48.2%)
<i>cochange.parquet</i> (175,204 rows)	
<code>repo_full, file_a, file_b</code>	String, no; repository and co-changed file pair
<code>support, lift, conf_*</code>	Int64 / Double, no; support count, lift, and directional confidences
<i>failure_messages.parquet</i> (3,490 rows)	
<code>run_id, job_id, test_id</code>	Int64 / String, no; run, job, and test identifiers
<code>failure_message</code>	String, no; captured textual failure message or stack trace

`docs/CLONE_PINS.json` rather than volatile branch heads. Basename-only matches achieve 0.5 confidence, while fully qualified class names that match uniquely achieve 1.0 confidence.

III. STORAGE AND SCHEMA

BR-BENCH is distributed in Apache Parquet format for typed schema enforcement and columnar efficiency, documented via a dataset datasheet [11]. Table I details the schema across release tables. Published release v0.1 [12] contains four tables: `instances` (165,349 rows), `outcomes` (12,766 rows), `cochange` (175,204 rows), and `failure_messages` (3,490 rows).

Release v0.2 (built offline and validated against schema.json via validator T1.6a, to be deposited as an updated version of the same Zenodo record before conference submission) maintains identical row counts and table grains while providing richer analytical fields. In `instances`, `base_run_id` is populated for 7,061 resolved runs (null for unresolved runs), with `base_status` tracking resolution mode across all 165,349 runs (152,768 `not_attempted`, 5,520 `no_base`, 4,245 `exact_green`,

TABLE II
PIPELINE ATTRITION FUNNEL ACROSS STAGES

Attrition Stage	Count	% Prev.	% Total
<i>Repositories</i>			
SEART GitHub export	3,671	—	100.00%
CI-live (≥ 100 runs in 90 days)	2,646	72.08%	72.08%
Test-intent workflow detected	2,333	88.17%	63.55%
Sampled into candidate frame	300	12.86%	8.17%
Swept by harvester	76	25.33%	2.07%
Repositories with ≥ 1 failed run	69	90.79%	1.88%
Repositories with ≥ 1 strict instance	37	53.62%	1.01%
<i>Workflow Runs</i>			
Total runs discovered	165,349	—	100.00%
Failed runs	12,581	7.61%	7.61%
Failed-job log captured on disk	6,586	52.35%	3.98%
Parsed head test failure	1,774	26.94%	1.07%
Resolved base run found	1,348	75.99%	0.82%
Known base failure set	897	66.54%	0.54%
Emitted strict benchmark instances	762	84.95%	0.46%

1,858 exact, 591 ancestor, 367 branch_prior), and base_run_distance recorded for 6,025 runs (median 0, p90 2, p99 8, max 10). Workflow start times are stored as typed UTC TIMESTAMP values in run_started_at. Changeset metrics (n_files_changed, n_lines_changed, touches_test_file, touches_build_config, touches_ci_config, is_docs_only, and changed_files file paths) are provided directly, set to null for the 17,364 runs lacking changeset diffs.

In outcomes, data is structured in a long grain by split (one row per instance $\times$ test $\times$ split, spanning 4,384 distinct pairs). Release v0.2 augments this table with bound source paths (test_file, 96.5% coverage), label_source, parser_confidence, status_head, n_job_rows (≥ 1 for 690 pairs), binding_status, binding_confidence (1.0 for FQCN, 0.5 for basename), duration_s (57.1% coverage), and failure_message_hash (48.2% coverage), while removing the pandas index artifact (__index_level_0__).

To protect contributor privacy, all author usernames in instances.parquet are pseudonymised using HMAC-SHA256 keyed by an operator-held secret, truncated to 16 hex characters across 1,791 unique authors (docs/DATASHEET.md §4). Failure messages were scanned for credentials; zero blocking secrets were identified, while 5 review samples were flagged for human verification.

IV. DATASET OVERVIEW AND QUALITY

A. Corpus and Attrition Funnel

The progressive filtering of repositories and workflow runs is summarized in Table II. Of the 165,349 workflow runs swept across 76 repositories, 12,581 (7.61%) concluded with a failure status. Among these, 6,586 had raw job logs preserved on disk, and 1,774 parsed to at least one test failure. Base-run matching resolved 1,348 of these runs, of which 897 possessed a known base failure set (either via an exact_green run or a parsed base log). Filtering out flakiness and runs lacking test output yielded 762 qualified benchmark instances in the strict split.

B. Language and Split Distribution

Table III summarizes the benchmark splits. The strict benchmark split comprises 762 pull-request instances and

TABLE III
BREAKDOWN BY SPLIT AND PROGRAMMING LANGUAGE

Split	Inst.	Labels	Tests	Java Inst.	Python Inst.
all	897	4,384	2,500	767 (85.5%)	130 (14.5%)
relaxed	770	4,214	2,484	646 (83.9%)	124 (16.1%)
strict	762	4,168	2,466	638 (83.7%)	124 (16.3%)

TABLE IV
TEST-TO-FILE BINDING AND QUALITY INDICATORS

Binding Strategy / Quality Metric	Count	Percentage
Full-confidence FQCN match ($c = 1.0$)	3,820	63.83%
Basename-only match ($c = 0.5$)	1,823	30.46%
Unbound: not found in source tree	178	2.97%
Unbound: ambiguous multi-match	164	2.74%
Combined exact binding	5,643	94.29%
Code-level assertions (regex lower bound)	729	17.49%
Timeout failures	112	2.69%
Strict environment failures (GPU, network)	89	2.14%
Unknown error text / unparsed stack trace	3,238	77.69%

4,168 fault-revealing labels representing 2,466 distinct tests across 37 repositories. Java accounts for 83.73% of instances (638/762) and 92.49% of labels (3,855/4,168), while Python represents 16.27% of instances (124/762) and 7.51% of labels (313/4,168).

C. Test Binding and Data Quality

Table IV reports test binding and error classification. Across 5,985 evaluated (repo, test_id) pairs, 94.29% (5,643/5,985) bind successfully to source files. Full-confidence FQCN resolution accounts for 63.83% (3,820/5,985), while basename matching resolves 30.46% (1,823/5,985). Only 5.71% remain unbound (2.97% not found, 2.74% ambiguous).

An automated regex audit classified failure messages into four classes: code-level assertions accounted for 17.49% (729/4,168), timeout errors represented 2.69% (112/4,168), and strict environment failures (e.g., missing CUDA GPUs or network timeouts) accounted for 2.14% (89/4,168). A majority of labels (77.69%, 3,238/4,168) remain unknown due to uncaptured stack traces or unparsed error texts.

V. HOW THE DATA HAS BEEN USED: RQ1 EXAMPLE

To demonstrate how BR-BENCH serves as an empirical benchmark, we evaluate an example research question (RQ1): *How accurately do conventional change impact analysis proxies and historical baselines identify fault-revealing tests?*

A. Experimental Setup and Leakage Audit

We evaluate four test selection heuristics at budget $k = 10$: (1) *Co-change (All)*: trailing 365-day co-change partners from Git commits; (2) *Co-change (Test Only)*: co-change restricted to conventional test paths (src/test/, *Test.java, test_*.py); (3) *Changeset Baseline*: pull-request modified files; and (4) *Historical Frequency*: k most frequently failing bound test files from strictly earlier runs (strict labels, counted once per (test, instance)). To prevent temporal data leakage, co-change history was mined strictly prior to each run’s start timestamp. An audit revealed that naive static co-change ended after the instance run in 713/713 audited instances and touched a changed file in 640/713 instances, while an

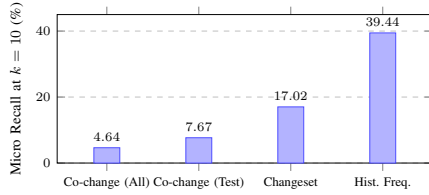


Fig. 2. Micro recall at $k = 10$ across test selection proxies evaluated on BR-BENCH ($n = 576$ identical instances, 1,316 positive labels).

uncorrected historical baseline double-counted failures across splits (224,758 rows for 78,399 distinct labels). Enforcing per-instance trailing windows, symmetric lookups, and single-split counting eliminated all leakage violations (0/713 in co-change and 0/762 in historical frequency).

B. Results

All methods are evaluated on identical instances where co-change yields predictions ($n = 576$; 1,316 positive test labels; 462 Java, 114 Python). As shown in Fig. 2, all-partner co-change achieves only 4.64% micro recall (61/1,316; mean P/R/J: 0.011 / 0.051 / 0.010; median size 14). Restricting co-change to conventional test files improves micro recall to 7.67% (101/1,316; mean P/R/J: 0.054 / 0.087 / 0.038; median size 4). Sensitivity testing with any path containing “test” yields identical recall (101/1,316, 7.67%; mean P/R/J: 0.048 / 0.087 / 0.033; median size 6). The changeset baseline attains 17.02% micro recall (224/1,316; mean P/R/J: 0.039 / 0.245 / 0.037; median size 9). In contrast, historical failure frequency achieves 39.44% micro recall (519/1,316; mean P/R/J: 0.185 / 0.517 / 0.178; median size 10). Removing environment and timeout failure labels leaves this ordering unchanged. Thus, historical failure frequency markedly outperforms version-control co-change heuristics at identifying true regressions.

VI. ORIGINALITY AND SIMILAR DATASETS

BR-BENCH differs from existing benchmarks along four axes: (1) **RTPTorrent** [7] provides Travis CI outcomes for Java by build job rather than PR diffs; it targets prioritization, not change-to-test linking; (2) **GHALogs** [8] archives raw logs without extracting test-level outcomes, linking PR diffs, resolving bases, or filtering flakiness; (3) **Static/Dynamic RTS Tools** (Ekstazi [3], STARTS [4], Chianti [13]) compute safe reachability over-approximations, whereas BR-BENCH captures empirical CI failures; and (4) **Industrial Studies** at Meta [5] and Google [6] deploy predictive selection on internal monorepos, whereas BR-BENCH is open and public.

VII. LIMITATIONS AND THREATS

We identify five key limitations: 1) *Roadmap Gates*: Gate 1 ($\geq 5,000$ strict positives) was **not met**, reaching 4,168/5,000 labels (83.36%) and 762/5,000 instances (15.24%). Gate 1.5 ($\geq 70\%$ binding) was met on combined binding (94.29%, 5,643/5,985) but **not met** at full-confidence FQCN resolution (63.83%, 3,820/5,985). 2) *Parser Evaluation*: Precision on fixture (46/46) and holdout v1 (30/30) reflects development tuning sets; blind holdout v5 was **not scored** without independent human annotation. 3) *Observational Ground Truth*: Labels reflect observational CI runs rather than isolated reruns;

environment failures (2.14%) and timeouts (2.69%) remain present. 4) *90-Day Retention Cliff*: Because GitHub Actions purges logs after 90 days, 33.21% of failed runs (4,458/13,422) had expired at harvest across the 77-repo corpus (as of 2026-08-29); full reproduction relies on our archived raw store. 5) *Repository Skew*: The strict split is dominated by Java (35 Java vs. 2 Python), with high volume from active projects (e.g., Apache Beam, Stirling-PDF).

VIII. FUTURE RESEARCH QUESTIONS AND IMPROVEMENTS

BR-BENCH enables three key directions: (1) *ML Test Selection*: training neural models on code diffs to predict regressions without execution overhead; (2) *Independent Re-execution*: executing planned causal subsets in isolated containers to separate regressions from environmental flakes; and (3) *Multi-Language Expansion*: extending coverage to JavaScript, TypeScript, and Go.

IX. AVAILABILITY AND REPRODUCIBILITY

Source code is released under the MIT licence and release tables under CC BY 4.0; underlying repository code remains under original licences. The published v0.1 bundle is deposited on Zenodo [12] (DOI: 10.5281/zenodo.23250262) containing `instances`, `outcomes`, `cochange`, and `failure_messages` (165,349 / 12,766 / 175,204 / 3,490 rows). Release v0.2 is built offline and validated against `schema.json` (validator T1.6a), to be deposited as an updated version of the same Zenodo record prior to conference submission. Full source code is on GitHub and Zenodo [14] (DOI: 10.5281/zenodo.23250690). All dataset numbers regenerate offline via `make tables (BR_OFFLINE=1)` in ≈ 8 minutes; 817 automated tests (814 passed, 3 skipped) pass cleanly via `make test`.

X. CONCLUSION

We presented BR-BENCH, an open benchmark linking pull-request modifications to fault-revealing tests across Java and Python GitHub Actions pipelines. By resolving base runs, filtering flakiness, and binding tests to source files, BR-BENCH provides reproducible ground truth for regression test selection and impact analysis. Our evaluation demonstrates that historical test failure frequency markedly outperforms conventional co-change proxies, establishing an empirical baseline for future predictive test selection research.

ACKNOWLEDGMENT

Generative AI assistants (Google Antigravity and Anthropic Claude) were used to draft and edit parts of the text, check references, and assist in developing the artifact’s software. The authors reviewed, rewrote and verified all content. No AI system is used in data collection, labelling, or computing any reported number; all numbers regenerate via `make tables`.

REFERENCES

- [1] M. Borg, K. Wnuk, B. Regnell, and P. Runeson, “Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context,” *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 675–700, 2017.
- [2] Y. Huang, J. Jiang, X. Luo, X. Chen, Z. Zheng, N. Jia, and G. Huang, “Change-patterns mapping: A boosting way for change impact analysis,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2376–2398, 2022.
- [3] M. Gligoric, L. Eloussi, and D. Marinov, “Practical regression test selection with dynamic file dependencies,” in *Proc. 2015 Int. Symp. Softw. Test. Anal. (ISSTA)*. ACM, 2015, pp. 211–222.
- [4] O. Legunsen, A. Shi, and D. Marinov, “STARTS: STAtic regression test selection,” in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), Tool Demos*. IEEE, 2017, pp. 949–954.
- [5] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, “Predictive test selection,” in *Proc. 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2019, pp. 91–100.
- [6] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, “Taming Google-scale continuous testing,” in *Proc. 39th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2017, pp. 233–242.
- [7] T. Mattis, P. Rein, F. Dürsch, and R. Hirschfeld, “RTPTorrent: An open-source dataset for evaluating regression test prioritization,” in *Proc. 17th Int. Conf. Mining Softw. Repos. (MSR)*. ACM, 2020, pp. 385–396.
- [8] F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon, “GHALogs: Large-scale dataset of GitHub actions runs,” in *Proc. 22nd Int. Conf. Mining Softw. Repos. (MSR), Data and Tool Showcase Track*. IEEE, 2025, pp. 669–673.
- [9] O. Dabic, E. Aghajani, and G. Bavota, “Sampling projects in GitHub for MSR studies,” in *Proc. 18th Int. Conf. Mining Softw. Repos. (MSR)*. IEEE, 2021, pp. 560–564.
- [10] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, “DeFlaker: Automatically detecting flaky tests,” in *Proc. 40th Int. Conf. Softw. Eng. (ICSE)*. ACM, 2018, pp. 433–444.
- [11] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [12] Deepanshu, S. Singh, P. Vadhavkar, and Yoga Raja C A, “BR-Bench: Execution-grounded change-to-failing-test data from GitHub actions (release v0.1),” <https://doi.org/10.5281/zenodo.23250262>, 2026.
- [13] X. Ren, F. Shah, F. Tip, B. G. Ryder, and O. Chesley, “Chianti: A tool for change impact analysis of Java programs,” in *Proc. 19th ACM SIGPLAN Conf. Object-Oriented Program., Syst., Lang., Appl. (OOPSLA)*. ACM, 2004, pp. 432–448.
- [14] Deepanshu, S. Singh, P. Vadhavkar, and Yoga Raja C A, “Blastradius and BR-Bench: Execution-grounded change impact data from GitHub actions,” <https://doi.org/10.5281/zenodo.23250690>, 2026.